Dictionaries

~ G. Anushiya Rachel
Assistant Professor, CSE
Shiv Nadar University Chennai

Introduction to Dictionaries

- Contains a pair of items – key and value
- The keys must be unique.
- Key and value are separated by a colon (:).
- Each pair is separated by a comma and the items are enclosed within{ }
- Dictionaries are unordered and mutable. Elements or values are accessed using keys instead of indices.
- Dictionaries can replace elif ladder/switch-case
- Example:

```
>>> d = {‘Alice’ : ‘F’, ‘Bob’ : ‘M’}
```

Creating Dictionary & Accessing Values

- Empty dictionary

```
>>> d1 = {}
```

- Dictionary with elements

```
>>> d2 = {1 : 'chocolate', 2 : 'vanilla', 3 : 'strawberry'}
>>> d3 = {'MS' : 'Python', 'CSE' : 'C++'}
>>> d2[1]
'chocolate'
>>> d3['CSE']
'C++'
```

Other Ways of Creating Dictionaries

- Creating by assigning keys dynamically

```
>>> d  = {}
>>> d['name'] = Bob
>>> d['age'] = 40
>>> d
{'name' : 'Bob', 'age' : 40}
```

- Creating by assignment

```
>>> dict(name = 'Bob', age = 40)
```

- Creating from tuples

```
>>> dict([('name', 'Bob'), ('age', 40)])
```

Adding New & Updating Existing Values

- Adding a new value

```
>>> d = {1: “hello”, 2 : “salut”}
>>> d[3] = ‘hola’
>>> d
{1: “hello”, 2 : “salut”, 3 : “hola”}
```

- Updating existing value

```
>>> d[2] = ‘bye’
>>> d
{1: “hello”, 2 : “bye”, 3 : “hola”}
```

Deletion

- Remove one element using ***del***

```
>>> d = {1 : 2, 3 : 4}
>>> del d[1]
>>> d
{3:4}
```

- Remove all elements using ***clear***

```
>>> d.clear() # Results in an empty dictionary
```

- Delete the dictionary

```
>>> del d      # Will delete the dictionary, causing it to no longer exist
```

More Operations / Methods

- No. of items in a dictionary using ***len***

```
>>> d = {1 : 2, '2' : 45}
>>> len(d)
2
```

- Membership test (availability of a key) using ***in***

```
>>> '2' in d
True
```

- Create a list of keys or values

```
>>> list(d.keys())          # Result: [1, '2']
>>> list(d.values())        # Result: [2, 45]
```

More Operations / Methods

- Create a list of items (key-value pairs) in a dictionary

```
>>> list(d.items())
[(1, 2), ('2', 45)]
```

- Get value corresponding to a key using ***get***

```
>>> d.get(1)
2
>>> d.get('h')
None
```

- Clone a list using ***copy***

```
>>> d1 = d.copy()    # d1 = d → aliasing i.e. changes to 1 will reflect in
                                the other as well
```

More Operations / Methods

- Update a dictionary using the ***update*** method

```
>>> D = {‘eggs’: 3, ‘spam’: 2, ‘ham’: 1}
>>> D1 = {‘toast’:4, ‘muffin’:5}
>>> D.update(D1)
>>> D
 {‘eggs’: 3, ‘spam’: 2, ‘ham’: 1, ‘toast’:4, ‘muffin’:5}
```

- Delete and return the value corresponding to key using ***pop***

```
>>> D.pop(‘spam’)
2
>>> D
 {‘eggs’: 3, ‘ham’: 1, ‘toast’:4, ‘muffin’:5}
```

Iterate through a Dictionary

```
>>> item_prices = {‘pencil’: 10, ‘pen’: 20, ‘eraser’: 5}
>>> for i in item_prices:
        print(f ‘Item: {i}’)
        print(f ‘Price: {item_prices[i]}’)Item: pencil
>>> Item: pencil
    Price: 10
    Item: pen
    Price: 20
    Item: eraser
    Price: 5
```

Reverse Look-Up

- Finding the value, given a key and a dictionary – look-up
- Finding the key, given a value – reverse look-up
  - There may be multiple keys corresponding to a given value.
    - Either choose one of the keys or make a list of all the keys.
  - There is no simple syntax for reverse look-up.
    - Search through the dictionary.
  - Reverse look-up is slower than a forward look-up.

Comprehensions

```
>>> D = {k: v for (k, v) in zip(['a', 'b', 'c'], [1, 2, 3])}
>>> D
{'a': 1, 'b': 2, 'c': 3}
>>>{x: x ** 2 for x in [1, 2, 3, 4]}
{1:1, 2:4, 3:9, 4:16}
>>> {c.lower(): c + '!' for c in ['EGGS', 'SPAM', 'HAM']}
{'eggs': 'EGGS!', 'spam': 'SPAM!', 'ham': 'HAM!'}
>>>{k:0 for k in ['a', 'b', 'c']}
 {'a': 0, 'b': 0, 'c': 0}
```

Programming Exercises

1. Write a Python script to generate and print a dictionary that contains a number (between 1 and n) in the form $\left(x, \frac{x^2}{4}\right)$
2. Convert two given lists to a dictionary.
3. Merge two given dictionaries into one. Observe what happens when both dictionaries have the same key.
4. Combine two dictionary by adding values for common keys.

   Example: Input: d1 = {'a': 100, 'b': 200, 'c':300}

   d2 = {'a': 300, 'b': 200, 'd':400}

   Output: {'a': 400, 'b': 400, 'd': 400, 'c': 300}
5. Check if a given value exists in a given dictionary and print the corresponding key(s).

Programming Exercises

6. Rename a specified key of a dictionary.
7. Get the key of the minimum value in the dictionary.
8. Obtain a string from the user and create a dictionary with the characters of the string as keys and their corresponding counts as values.
9. Obtain the prices of N items from a user and create a list. Print the top k priciest items and their prices.
10. Sort a given dictionary by its keys.
11. Remove duplicate values (and the corresponding key) from a given dictionary.
12. Obtain the name and scores earned by N players and store them in the form of a dictionary. Based on the user's requirement, return the dictionary entries corresponding to the scores between 2 values, say a and b.

Programming Exercises

13. Convert a given dictionary of lists into lists of dictionaries.

    Example input: {'Science': [88, 89, 62, 95], 'Language': [77, 78, 84, 80]}

    Expected output: [{'Science': 88, 'Language': 77}, {'Science': 89, 'Language': 78}, {'Science': 62, 'Language': 84}, {'Science': 95, 'Language': 80}]

14. Iterate through a given dictionary and convert any number present in the form of a string into an integer or floating-point value appropriately.

15. Create a simple calculator using a dictionary in the place of conditional statements.